

基于高斯泼溅的移动端3D场景重建系统设计 与实现

结题报告草稿（XeTeX 排版版）

学生姓名：（填写姓名）

学号：（填写学号）

指导教师：（填写教师）

项目周期：2025 年 11 月 - 2026 年 6 月

完成时间：2026 年 6 月

本稿按照中期报告的研究叙述方式展开，保留了结题答辩所需的系统截图、训练结果、点云下载对比和演示流程图位。定稿时可将空图框替换为小程序、AutoDL 后端、点云下载页、Viewer 结果和训练日志截图。

摘要

本项目面向普通用户移动端三维内容生产需求，设计并实现了一套“微信小程序采集上传 + AutoDL 云端训练 + 浏览器查看与点云下载”的端云协同 3D 场景重建系统。系统采用 Spann3R 生成非标定图像序列的相机位姿和初始点云，再使用 Nerfstudio Splatfacto 完成 3D Gaussian Splatting 训练与结果导出。针对个人主体小程序不能使用 WebView、AutoDL 公开端口受限、系统盘空间有限、点云文件体积较大等工程约束，本项目在上传、训练、下载三个层面进行了可用性优化。

上传层面，系统通过 6008 管理入口下的 `/upload-proxy` 路径转发到内部上传服务，前端提供上传状态、失败重试、训练阶段提示和结果链接复制能力。训练层面，后端管理面板支持任务启动、停止、日志摘要、阶段进度和等待队列，单卡环境下默认串行调度训练任务，避免多个训练进程同时抢占显存。下载层面，系统在点云下载前加入空间裁切、体素下采样和缓存复用机制，默认下载处理后点云，并保留原始点云下载入口和多文件 ZIP 打包能力。经远程 AutoDL 实例验证，6008 健康检查、状态接口、点云清单、处理后单文件下载和处理后 ZIP 下载均可用；示例点云由约 482 KB 处理为约 17 KB，移动端下载压力明显降低。

关键词：移动端三维重建；高斯泼溅；Spann3R；Splatfacto；AutoDL 云端；微信小程序；点云下采样

Contents

1	研究背景与项目目标	3
1.1	研究背景	3
1.2	工程约束	3
1.3	项目目标	3
2	系统总体设计	4
2.1	端云协同架构	4
2.2	6006/6008 路径方案	5
2.3	文件与存储策略	6
3	上传层优化	6
3.1	小程序上传体验	6
3.2	后端传入文件处理	7
4	训练层优化	7
4.1	Spann3R + Splatfacto 流程	7
4.2	单卡任务队列	8
5	下载层优化	9
5.1	点云发现与分类	9
5.2	空间裁切与体素下采样	9
5.3	多文件下载与原始文件保留	9
6	前后端 UI 优化	10
6.1	小程序端	10
6.2	后端管理 UI	10
7	验证结果	11
7.1	接口与运行状态验证	11
7.2	优化前后对比	12
8	同卡对比 workflow 部署与短基准	12
9	结题材料截图位	14
10	总结与展望	14
	参考文献	16

1 研究背景与项目目标

1.1 研究背景

随着数字文旅、线上商品展示、校园场景复现、教育演示和社交内容传播的发展，三维场景不再只服务于专业建模团队，而逐渐成为普通用户也需要创建、查看和分享的数字内容。传统三维建模依赖专业设备、标定相机或人工建模经验，制作周期长、成本高；经典 SfM/MVS 方法虽然几何基础成熟，但通常要求较规范的采集条件和离线处理经验；NeRF 类方法在新视角合成质量上表现突出，但训练和渲染成本较高，不利于移动端交互。

3D Gaussian Splatting 通过显式高斯基元表示三维场景，在视觉质量和实时渲染之间取得了较好平衡，适合作为移动端可视化展示的结果形态。与此同时，Spann3R、DUST3R 等非标定重建方法能够从普通图像序列中快速得到相机位姿和初始点云，为后续 3DGS 训练提供几何先验。因此，将 Spann3R 与 Splatfacto 组合，并通过服务化后端连接微信小程序和云端 GPU，是本项目的核心技术路线。

1.2 工程约束

本项目不是单纯运行一个重建算法，而是需要把算法能力转化为普通用户可操作的系统闭环，主要约束如下：

- **移动端平台约束：**个人主体微信小程序无法使用 WebView 内嵌网页，因此 Viewer 和下载页需要保持“复制链接/跳转浏览器或电脑端打开”的引导方式。
- **端口约束：**AutoDL 实例没有独立公网 IP，官方文档说明实例的 6006 和 6008 端口会映射到公网访问地址；多个 HTTP 服务可通过路径代理转发到不同内部端口。
- **存储约束：**AutoDL 文档说明 /root/autodl-tmp 是不占用系统盘空间的特殊目录，默认租用实例提供 50GB 数据盘；训练数据、上传照片和下载缓存应尽量放在数据盘，避免系统盘堆满。
- **显存约束：**Spann3R 与 Splatfacto 同在单张 GPU 上运行时，需要避免并行训练导致显存竞争，因此训练任务需要排队或串行调度。
- **下载约束：**原始点云和 Gaussian 导出点云可能较大，直接下载整个点云会影响移动端体验，因此需要默认提供裁切、下采样后的轻量结果。

1.3 项目目标

结题阶段目标是在已有中期成果基础上形成一个可演示、可复核、可交付的端到端系统：

1. 打通小程序照片采集、上传、后端状态查看、训练触发、结果下载和 Viewer 查看流程。
2. 在 AutoDL 6006/6008 限制下完成路径级服务整合，保证上传、管理、下载、Viewer 入口清晰。
3. 优化上传体验，降低用户对后台状态和训练阶段的理解成本。
4. 优化训练调度，在单卡条件下避免多任务直接并发造成显存冲突。
5. 优化点云下载，默认提供空间裁切、体素下采样和多文件打包能力。
6. 形成结题报告、答辩 PPT、接口文档、用户手册和截图清单，支撑答辩展示。

项目总体闭环流程图

此处替换为实际系统截图或结果图

建议截图后保持 16:9 或 4:3 比例，避免拉伸变形

Figure 1: 项目总体闭环流程图

2 系统总体设计

2.1 端云协同架构

系统由前端小程序、AutoDL 后端服务、算法训练流程和结果交付层组成。小程序负责移动端采集、状态查看和链接引导；后端服务负责上传接收、任务管理、配置管理、日志解析、点云发现和下载处理；算法流程负责 Spann3R 重建、Nerfstudio Splatfacto 训练和 Gaussian 点云导出；结果交付层负责 Viewer、点云下载页和多文件 ZIP 打包。

Table 1: 系统模块划分

模块	职责
微信小程序前端	拍摄/选择照片、上传进度、训练状态、点云清单、链接复制与外部打开引导。
上传服务	接收图片、校验扩展名和大小、生成会话文件名、写入上传清单。
管理面板	提供 6008 首页、健康检查、任务启动/停止、配置管理、日志摘要、队列状态。
训练流程	快照输入照片、运行 Spann3R、转换 Nerfstudio 数据、启动 Splatfacto、导出 Gaussian。
点云下载服务	发现点云、按场景/类型筛选、处理后下载、原始下载、ZIP 打包、缓存清理。
Viewer	在 6006 或浏览器外链中展示 Nerfstudio/3DGS 可视化结果。

2.2 6006/6008 路径方案

结合用户补充的“不同 path 转发到本地不同端口”思路，系统将 6008 作为管理、状态、上传代理和下载入口，将 6006 保留给 Viewer。上传服务实际可运行在内部端口，例如 127.0.0.1:7006，外部通过 6008/upload-proxy/upload 访问；点云下载、状态接口和管理 UI 直接运行在 6008 管理服务中。

Table 2: 端口与路径规划

公网入口	路径	用途
6008	/	后端管理 UI 首页，展示训练阶段、日志、配置和结果入口。
6008	/ healthz、/ api/status	健康检查和训练状态。
6008	/ upload-proxy/upload	小程序上传照片入口，转发到内部上传服务。
6008	/ downloads、/ download/...	点云下载页、处理后下载、原始下载和 ZIP 打包。
6006	/	Viewer 或训练可视化页面。

AutoDL 6006/6008 路径转发方案截图

此处替换为实际系统截图或结果图
建议截图后保持 16:9 或 4:3 比例，避免拉伸变形

Figure 2: AutoDL 6006/6008 路径转发方案截图

2.3 文件与存储策略

为避免系统盘堆积，本项目将训练相关文件集中放在 `/root/autodl-tmp` 数据盘下。上传照片位于 `/root/autodl-tmp/input_images`，场景数据位于 `/root/autodl-tmp/gs_train/scenes`，处理后点云缓存位于 `/root/autodl-tmp/pointcloud_processed_cache`。缓存目录设置最大保留时间和最大文件数，历史场景和测试照片集也设置保留上限。

3 上传层优化

3.1 小程序上传体验

上传体验的核心是让用户知道“当前能不能上传、上传到哪里、上传是否成功、后端是否开始处理”。本次优化在预览页加入后端健康状态、上传统计、训练阶段提示、任务队列信息、上传目录摘要和点云清单，使小程序不再只是单向上传工具，而是端到端状态面板。

- 上传入口从直接访问内部服务改为访问 `6008/upload-proxy/upload`，适配 AutoDL 公开端口限制。
- 上传前进行状态检查，后端不可用时给出明确提示，减少盲目上传。
- 上传结果支持失败统计和重试提示，避免用户误以为全部文件已成功提交。
- 预览页显示 Viewer、管理面板、下载页等链接，但不使用 WebView 内嵌，符合个人小程序限制。

微信小程序上传与状态页面截图

此处替换为实际系统截图或结果图

建议截图后保持 16:9 或 4:3 比例，避免拉伸变形

Figure 3: 微信小程序上传与状态页面截图

3.2 后端传入文件处理

上传服务对文件扩展名、大小、会话编号和帧序号进行处理。文件以会话编号、帧序号、时间戳和随机后缀命名，避免同名覆盖；写入过程先保存为临时 `.part` 文件，成功后再替换为正式文件，减少半文件进入训练流程的概率；同时写入上传清单，便于后续统计和问题追踪。

该策略提升了三方面稳定性：第一，非法格式不会进入训练目录；第二，异常中断的半文件不会被误认为可训练图片；第三，上传清单可作为结题阶段“实际上传链路存在”的证据。

4 训练层优化

4.1 Spann3R + Splatfacto 流程

训练流程分为两阶段。第一阶段使用 Spann3R 从移动端图像序列中恢复相机位姿和初始点云，解决非标定输入缺少几何先验的问题；第二阶段将 Spann3R 输出转换为 Nerfstudio 可用数据结构，交给 Splatfacto 训练高斯场景，并在训练完成后导出点云或 Gaussian 相关结果。

该组合相比直接使用传统 COLMAP 初始化，优势在于对普通手机拍摄序列更友好，流程自动化程度更高；相比只输出 Spann3R 点云，Splatfacto 能持续提升新视角合成效果，便于 Viewer 展示。



Figure 4: Spann3R 与 Splatfacto 两阶段训练流程图

4.2 单卡任务队列

用户提出可以考虑多任务同时训练，但在单张 4090/4090D 环境中，多个 Splatfacto 训练任务同时运行很容易造成显存竞争和 Viewer 端口冲突。因此本次优化采用“多任务可排队、训练默认串行”的策略：当前任务运行时，新的启动请求进入等待队列；当前任务结束后，队列任务自动接续。这样既支持多次点击或多批任务提交，又避免真正并行训练导致系统不稳定。

Table 3: 训练调度策略

场景	处理方式
无任务运行	立即启动训练流程，并记录 active job。
已有任务运行	新任务进入队列，管理面板和小程序显示队列长度。
用户停止任务	停止当前训练，并清空等待队列，避免旧任务误启动。
Viewer 端口占用	训练前检查并终止冲突进程，保证 6006 留给当前 Viewer。



Figure 5: 后端训练队列与日志面板截图

5 下载层优化

5.1 点云发现与分类

下载服务会扫描 `/root/autodl-tmp/gs_train`、`/root/autodl-tmp/Spann3R/output/demo` 等目录，发现 `.ply` 点云文件并按场景、类型和修改时间排序。文件类型包括 `raw`、`downsampled`、`train`、`gaussian` 和 `other`，前端和管理面板均可展示点云清单。

5.2 空间裁切与体素下采样

原始点云直接下载会带来两个问题：文件体积大，且包含背景点、远距离离群点和训练无关区域。为此，系统在处理后的下载中加入两步优化：

1. **空间裁切：**若同一场景存在下采样参考点云，则读取其空间边界框，并按一定 `padding` 比例扩展，裁掉超出主体范围的点。
2. **体素下采样：**对裁切后的点云执行 `voxel down sample`，降低点数量和文件大小。

处理后文件写入缓存目录，并将源文件状态、参考文件状态、体素大小和 `padding` 比例写入缓存 `key`。相同参数再次下载时直接复用缓存，减少重复计算。

点云下载页与处理参数截图

此处替换为实际系统截图或结果图

建议截图后保持 16:9 或 4:3 比例，避免拉伸变形

Figure 6: 点云下载页与处理参数截图

5.3 多文件下载与原始文件保留

系统默认下载处理后点云，但仍保留 `processed=false` 参数用于下载原始点云。多文件下载通过 ZIP 打包实现，可按场景、类型或文件 `id` 筛选。ZIP 内部路径按“场景/类型/文件名”组织，便于答辩和实验复核时查看多个输出版本。

Table 4: 点云下载能力

能力	说明
最新点云下载	/download/latest?prefer=..., 按类型优先级选择最新文件。
处理后点云下载	/download/processed/latest 或默认 processed=true。
原始点云下载	在下载链接追加 processed=false。
按 ID 下载	/download/{file_id}, 用于小程序点云清单。
多文件 ZIP	/download/zip?variant=downsampled&processed=true。
缓存清理	按最大保留时长和最大文件数自动清理处理后点云。



Figure 7: 原始点云与优化后点云大小对比截图

6 前后端 UI 优化

6.1 小程序端

小程序预览页从单一“查看链接”扩展为后端状态面板。页面展示上传门控健康、6008 健康状态、上传统计、训练进程、等待队列、当前阶段、上传目录摘要、场景摘要和点云清单。用户可以复制上传接口、Viewer 地址、后端 UI、点云下载页、最新点云、优化后点云和 ZIP 下载链接。

因为个人主体小程序不能内嵌 WebView，系统没有把 Viewer 强行嵌入小程序，而是保留“复制链接到浏览器或电脑端打开”的方案，并在页面文字中明确说明。

6.2 后端管理 UI

6008 后端管理 UI 负责集中展示运行态，包括：

- 训练是否运行、进程 PID、当前任务和等待队列。
- 当前阶段：输入监听、Spann3R、Gaussian 训练、完成或异常。
- 上传目录、场景目录、点云数量和日志摘要。

- 配置项编辑，包括最少图片数、轮询间隔、点云下采样参数和缓存策略。
- 点云下载页入口与清理按钮。



Figure 8: AutoDL 6008 后端管理 UI 截图

7 验证结果

7.1 接口与运行状态验证

截至本稿生成时，远程 AutoDL 实例已完成以下验证：

Table 5: 后端验证记录

验证项	结果	说明
6008 健康检查	通过	/healthz 返回 200，服务状态为 ok。
训练状态接口	通过	/api/status 返回运行状态、PID 和队列长度。
点云清单接口	通过	/api/pointclouds/summary 发现 10 个历史点云文件。
处理后单文件下载	通过	示例 downsampled 点云由约 482 KB 处理为约 17 KB。
处理后 ZIP 下载	通过	示例 ZIP 包含 4 个处理后点云文件。
本地语法检查	通过	上传服务、下载服务、管理面板和训练流程 Python 文件均通过语法检查。

7.2 优化前后对比

Table 6: 可用性优化前后对比

层面	优化前	优化后
上传	用户不易判断后端是否处于可上传阶段。	小程序显示 6008 健康、上传统计、阶段提示，并通过 /upload-proxy 进入内部上传服务。
训练	启动训练容易与已有进程或 Viewer 端口冲突。	管理面板维护 active job 和等待队列，单卡默认串行调度。
下载	直接下载完整点云，文件体积和移动端压力较大。	默认裁切/下采样并缓存处理后点云，支持原始下载和 ZIP 打包。
文件管理	上传、输出和缓存可能分散在系统盘或旧目录。	统一使用 /root/autodl-tmp 数据盘，并设置缓存/历史保留限制。
UI	小程序与后端面板信息割裂。	小程序展示训练状态、队列、上传摘要和点云清单；后端 UI 提供配置、日志、下载入口。



Figure 9: 系统验证截图：健康检查、点云清单和下载结果

8 同卡对比 workflow 部署与短基准

为在答辩中体现 Spann3R + Splatfacto 架构的优势，本项目在同一张 RTX 4090 上建立了隔离对比工作区。对比实验目录为 /root/autodl-tmp/workflow_benchmarks，位于 AutoDL 数据盘中，不修改正在运行的 6008 后端服务，也不直接污染 base conda 环境。实验前数据盘约剩余 23GB，短基准完成后该目录约占用 284MB，数据盘仍约剩余 22GB。

对比工作区使用已有场景 `scene_20260418_151918_83bf1769`，该场景包含 64 张图、`transforms.json`、约 11.78MB raw 点云和约 482KB downsampled/init 点云。另复制 24 张小样本到 `workflow_benchmarks/sample_images`，用于后续传统 COLMAP 基线。当前环境中 `ns-train` 与 `ns-process-data` 可用，但 `colmap` 不在 PATH 中，因此未直接安装传统 COLMAP，避免影响原环境。

Table 7: 同卡短基准结果

工作流	迭代数	结果	说明
Nerfacto	20	成功，16 秒	证明当前 Nerfstudio 训练框架可启动；日志提示缺少 <code>tiny-cuda-nn</code> ，速度不是最佳。
Splatfacto	20	首次失败，12 秒	失败原因为 <code>gsplat</code> CUDA 扩展无法加载，缺少 <code>ninja</code> 。
Splatfacto 隔离修复	隔 20	成功，230 秒	将 <code>ninja</code> 安装到 <code>workflow_benchmarks/pydeps</code> ，并将 Torch 扩展缓存放到数据盘；首次编译 CUDA 扩展耗时较长。
COLMAP Splatfacto	+ 未完整运行	暂缓	<code>colmap</code> 不在 PATH 中，正式对比前建议单独安装到隔离 conda 环境。

短基准表明，其他工作流可以在同一张卡上部署，但必须做好环境隔离。Nerfacto 可以作为神经渲染基线，Splatfacto 是主流程的高斯训练核心；传统 COLMAP 基线则需要额外安装和维护。相比直接依赖传统 COLMAP，Spann3R + Splatfacto 的优势在于先用 Spann3R 生成非标定手机输入的位姿和初始点云，再进入 Splatfacto 训练，从而降低普通手机拍摄数据进入 3DGS 流程的门槛。下载层的裁切与下采样进一步保证结果不仅能训练，也更适合移动端下载和答辩展示。



Figure 10: 同一数据集下不同方案短基准与效果对比图

9 结题材料截图位

以下截图建议在最终定稿时补齐：

1. 小程序采集页：展示拍摄/选择照片、上传入口和引导提示。
2. 小程序预览页：展示上传统计、训练状态、等待队列和点云列表。
3. 6008 后端管理面板：首页状态、训练阶段、配置项和日志摘要。
4. 6008 点云下载页：展示文件分类、优化下载、原始下载和 ZIP 打包。
5. 处理后下载对比：展示 raw 文件大小与 processed 文件大小。
6. Viewer 结果页：展示训练结果或已有点云/高斯结果的可视化效果。
7. AutoDL 目录截图：展示 `/root/autodl-tmp` 下的输入、场景、缓存目录。
8. 训练日志截图：展示 Spann3R、Splatfacto、Gaussian 导出的关键阶段。

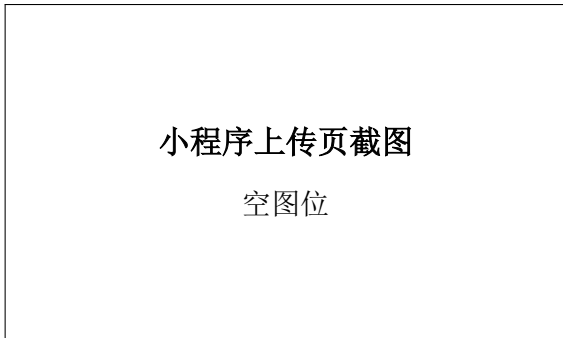


Figure 11: 小程序上传页截图

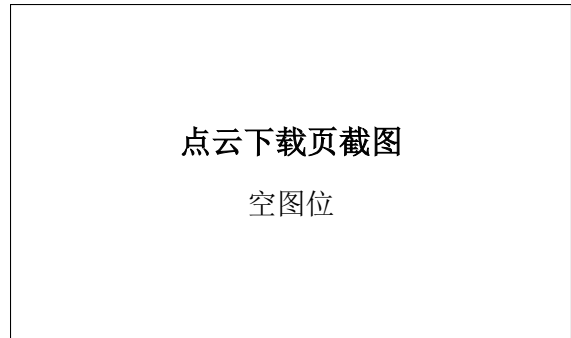


Figure 12: 点云下载页截图



Figure 13: Viewer 结果截图



Figure 14: 训练日志截图

10 总结与展望

本项目在结题阶段完成了从移动端采集、云端训练到结果下载的系统化优化。上传层面，系统通过 6008 路径代理和小程序状态面板解决了端口受限与用户反馈不足的问题；训练层面，后端通过任务队列、日志解析和阶段状态降低了单卡训练的不确定性；下载层面，系统通过空间裁切、体素下采样、缓存复用和 ZIP 打包改善了点云文

件过大的体验问题。整体上，系统已经从中期阶段的“算法链路可跑”推进到结题阶段的“前后端可联调、流程可展示、结果可下载、材料可复核”。

后续仍有三方面可继续扩展。第一，补充更多真实场景数据，并形成量化对比，例如训练耗时、点云数量、下载文件大小、Viewer 帧率和主观视觉质量。第二，在当前隔离工作区基础上补齐 COLMAP + Splatfacto 完整基线，并与 Spann3R-only、Spann3R + Splatfacto 形成同卡对照，以更直观地说明主流程优势。第三，若后续具备企业主体小程序权限，可将 Viewer 访问方式升级为更完整的端内或半端内体验；当前个人主体场景下，继续保留外部浏览器查看是更稳定且合规的方案。

参考文献

- [1] Kerbl B., Kopanas G., Leimkühler T., Drettakis G. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics, 2023.
- [2] Tancik M., Casser V., Yan X., et al. Nerfstudio: A Modular Framework for Neural Radiance Field Development. SIGGRAPH, 2023.
- [3] Wang H., Agapito L. 3D Reconstruction with Spatial Memory. arXiv, 2024. <https://arxiv.org/abs/2408.16061>
- [4] DUS3R: Geometric 3D Vision Made Easy. <https://github.com/naver/dust3r>
- [5] Nerfstudio Splatfacto Documentation. <https://docs.nerf.studio/nerfology/methods/splat.html>
- [6] AutoDL 帮助文档：开放端口。AutoDL 官方说明实例 6006 和 6008 可映射到公网访问地址。 <https://www.autodl.com/docs/port/>
- [7] AutoDL 帮助文档：暴露多个服务。AutoDL 官方说明可通过路径路由转发到不同内部服务。 https://www.autodl.com/docs/proxy_in_instance/
- [8] AutoDL 帮助文档：本地数据盘与系统盘空间不足。 https://www.autodl.com/docs/local_disk/; <https://www.autodl.com/docs/qa1/>